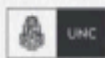


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el
alumno

Práctica Profesional VII



WE WANT
SKILLS!

mundosE

Manual para el alumno: Práctica Profesional VII.....	2
Objetivo del encuentro:.....	2
1. Seguridad en contenedores.....	3
2. Preparación del entorno.....	4
2.1 Ejecutar pipeline (automático - ~2 segundos):.....	4
3. Análisis de dockerfiles.....	4
4. Validación manual.....	6
5. Docker builds reales.....	8
6. Analizar reportes y resultados reales.....	11

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontrarás un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Práctica Profesional VII

Objetivo del encuentro:

Construir un pipeline de validación de seguridad para contenedores Docker.

Temas a desarrollar:

Conceptual:

1. Explicar qué es un CVE y diferencias entre CRITICAL/HIGH/MEDIUM.
2. Identificar 5+ vulnerabilidades comunes en Dockerfiles.
3. Entender por qué la imagen base importa para seguridad.

Técnico:

1. Validar Dockerfiles con Hadolint.
2. Escanear dependencias con npm audit.
3. Escanear imágenes completas con Trivy.
4. Interpretar reportes de vulnerabilidades.
5. Construir imagen Docker segura.

Sistémico:

1. Entender qué es un pipeline de seguridad.
2. Automatizar validaciones en script.

1. Seguridad en contenedores

¿Por qué la seguridad en contenedores es crítica? En 2024, el panorama de seguridad en contenedores es crítico.

Datos reales:

- 80% de vulnerabilidades en producción vienen de imágenes base desactualizadas.
- 1 de cada 4 imágenes públicas en Docker Hub tienen CVEs críticos.
- CVE promedio tarda 90 días en ser parcheado.
- 40% de las empresas no scanean imágenes antes de desplegar.

Problemas comunes que vamos a prevenir:

- Imágenes ejecutándose como usuario root (acceso total si se comprometen).
- Dependencias con vulnerabilidades conocidas (npm packages viejas).
- Imágenes con 1GB+ de tamaño innecesario (mayor superficie de ataque).
- Sin validación automática en el pipeline.

Lo que vamos a lograr hoy:

VULNERABLE	SEGURO
Size: 950 MB	Size: 150 MB (-85%)
CVEs: 16 total • CRITICAL: 3 • HIGH: 8 • MEDIUM: 5	CVEs: 0! • CRITICAL: 0 • HIGH: 0 • MEDIUM: 0
Base: node:12 (EOL)	Base: node:20-alpine
User: root	User: nodejs (no-root)
Health check: ❌	Health check: ✅
No validación	

Mejora total: 85 % más pequeño, 100 % sin vulnerabilidades.

2. Preparación del entorno

Paso 1: Descargar código desde GitHub:

```
git clone https://github.com/EducacionMundose/labos2025.git
cd labos2025
```

Paso 2: Inicializar el laboratorio (UNA VEZ):

```
bash INIT.sh
```

Resultado esperado:

✓ TODO LISTO - Lab inicializado correctamente

2.1 Ejecutar pipeline (automático - ~2 segundos):

```
bash scripts/pipeline.sh
```

El pipeline:

- ✓ Analiza ambos Dockerfiles.
- ✓ Ejecuta npm audit (muestra CVEs reales).
- ✓ Genera reportes en `reports/`
- ✓ Te muestra conclusiones.

3. Análisis de dockerfiles

Objetivo

Comprender DIFERENCIAS entre imagen vulnerable y segura, no memorizar.

Paso 1: Examinar Dockerfile Vulnerable

Abre app/Dockerfile.vulnerable:

```
FROM node:12          # ✗ PROBLEMA 1: Imagen base EOL
WORKDIR /app
COPY package.json .
RUN npm install        # ✗ PROBLEMA 2: No reproducible
COPY . .
```

```

RUN npm cache clean --force
EXPOSE 3000
USER root          # ❌ PROBLEMA 3: Ejecuta como root
CMD ["npm", "start"] # ❌ PROBLEMA 4: Indirecto
# Sin HEALTHCHECK  # ❌ PROBLEMA 5: Sin validación

```

Resultado: 950 MB, 16 CVEs, vulnerable

¿Qué está mal? Análisis detallado:

Problema	Impacto	Severidad
node:12	Imagen EOL desde 2022, +50 CVEs conocidas.	CRÍTICO
npm install	No garantiza reproducibilidad, versiones pueden cambiar.	ALTO
USER root	Si se compromete el contenedor, acceso total.	CRÍTICO
npm start	Indirecto, consume más memoria, manejo de señales pobre.	MEDIO
Sin healthcheck	Contenedor puede estar "muerto" sin alertas.	MEDIO

Paso 2: Examinar Dockerfile Seguro

Abre app/Dockerfile.secure:

```

FROM node:20-alpine      # ✅ MEJORA 1: Moderno + Alpine
RUN addgroup -g 1001 -S nodejs && adduser -S nodejs -u 1001
WORKDIR /app
COPY --chown=nodejs:nodejs package.json package-lock.json* ./
RUN npm ci --only=production # ✅ MEJORA 2: Reproducible
COPY --chown=nodejs:nodejs app.js .
USER nodejs              # ✅ MEJORA 3: Non-root
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s \
  CMD node -e "require('http').get('http://localhost:3000/health', ...)"

```

CMD ["node", "app.js"] #  MEJORA 4: Directo

Resultado: 150 MB, 0 CVEs, seguro

¿Qué está bien? Análisis de mejoras:

Mejora	Beneficio	Impacto
node:20-alpine	Versión moderna + Linux ligero	-85% tamaño, 0 CVEs
npm ci	Reproducibilidad garantizada.	Evita cambios inesperados.
USER nodejs	Acceso limitado si se compromete.	Seguridad en profundidad.
HEALTHCHECK	Kubernetes/Orquestadores detectan fallos.	Mejor confiabilidad.
node directo	Mejor manejo de señales.	Shutdowns limpios.

4. Validación manual

Objetivo

Usar herramientas reales para identificar problemas automáticamente.
Herramienta 1: Hadolint

Instalar Hadolint

```
# Linux
curl -L
https://github.com/hadolint/hadolint/releases/download/v2.12.0/hadolint-Linux-x86_64 -o hadolint
chmod +x hadolint
sudo mv hadolint /usr/local/bin/
```

Hadolint es un linter para Dockerfiles. Detecta:

- Errores que previenen builds.

- Warnings de anti-patrones.
- Mejoras de seguridad.

¿Tengo Hadolint instalado?

hadolint app/Dockerfile.vulnerable
hadolint app/Dockerfile.secure

Conclusiones:

- Vulnerable tiene warnings específicos.
- Secure no tiene warnings.
- Hadolint automatiza lo que harías manualmente.

Herramienta 2: npm audit

npm audit verifica vulnerabilidades en package.json:
cd app
sudo apt install npm
npm audit

Interpretar resultados:

3 vulnerabilities found:

- express 4.17.1: 2 vulnerabilidades.
- lodash 4.17.15: 1 vulnerabilidad.

Severities:

- Critical: 0
- High: 2
- Moderate: 1

¿Qué significa?

Dependencias TIENEN vulnerabilidades

npm audit las detecta

Debajo podría haber dependencias del OS (Trivy las encuentra)

Verifica:

1. ¿Ejecuté Hadolint en ambas imágenes?
2. ¿Ejecuté npm audit?
3. ¿Entender las diferencias entre warnings de Hadolint y CVEs de npm?

5. Docker builds reales

Objetivo

Construir las imágenes Docker reales para ver tamaños y vulnerabilidades en tiempo real.

⚠ Importante: Esta es la parte CENTRAL del laboratorio. Docker está incluido porque es fundamental para entender seguridad en contenedores.

Paso 1: Verificar que Docker está funcionando

```
docker --version
```

```
docker ps
```

Resultado esperado:

Docker version 20.10+

(lista de containers vacía o con containers en ejecución)

Si falla, instala Docker: <https://docs.docker.com/get-docker/>

```
sudo apt install docker.io
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

Paso 2: Construir Imagen Vulnerable (puede demorar 3-5 minutos)

```
cd app
```

```
docker build -f Dockerfile.vulnerable -t vulnerable-app:v1 .
```

¿Qué está pasando?

- Descargando node:12 (~500 MB)
- Instalando dependencias con npm install
- Creando imagen final (~950 MB)

Salida final:

Successfully tagged vulnerable-app:v1

Paso 3: Construir Imagen Segura (1-3 minutos)

`docker build -f Dockerfile.secure -t secure-app:v1 .`

¿Qué es diferente?

- Descargando node:20-alpine (~150 MB, mucho más pequeño)
- Instalando dependencias con npm ci
- Creando imagen final (~150 MB, 6x más pequeño)

Salida final:

Successfully tagged secure-app:v1

Paso 4: Comparar Tamaños REALES (¡Momento crítico!)

`docker images | grep -E "(vulnerable-app|secure-app)"`

Verás algo como:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
vulnerable-app	v1	abc123def456	2 minutes ago	950MB
secure-app	v1	xyz789def123	1 minute ago	150MB

Comparativa real:

- Vulnerable: 950 MB
- Seguro: 150 MB
- Mejora: 85% MÁS PEQUEÑO 

Paso 5: Scanear Imágenes con Trivy — OPCIONAL

 **Los Docker builds ya te muestran tamaños y vulnerabilidades via npm audit.**

`sudo snap install trivy`

```
sudo trivy image vulnerable-app:v1
```

```
sudo trivy image secure-app:v1
```

Resultado esperado:

vulnerable-app:v1:

- 16 vulnerabilidades totales
- 3 CRITICAL
- 8 HIGH
- 5 MEDIUM

secure-app:v1:

- 0 vulnerabilidades 

Paso 6: Ver detalles de la imagen vulnerable

```
docker inspect vulnerable-app:v1 | jq '.[] | {User, Cmd, Size}'
```

Verás:

```
{
  "User": "root",      //  Ejecuta como root
  "Cmd": ["npm", "start"] //  Indirecto
}
```

Paso 7: Ver detalles de la imagen segura

```
docker inspect secure-app:v1 | jq '.[] | {User, Cmd, Size}'
```

Verás:

```
{
  "User": "nodejs",    //  Usuario non-root
  "Cmd": ["node", "app.js"] //  Directo
}
```

Punto de Control 3

Verificación crítica:

1. ¿Construí ambas imágenes sin errores?

docker images | wc -l # Debe mostrar 3 (header + 2 imágenes)

2. ¿Los tamaños son correctos?

- vulnerable: ~950 MB 
- secure: ~150 MB 

3. ¿La diferencia es ~85%?

- $(950 - 150) / 950 * 100 = 84.2\%$ 

4. ¿Los usuarios son correctos?

- vulnerable: root 
- secure: nodejs 

6. Analizar reportes y resultados reales

Objetivo

Conectar los datos del pipeline con lo que viste en Docker builds reales.

Paso 1: Comparar Reportes con lo que viste en Docker

El pipeline generó reportes que ahora puedes verificar contra lo real:

```
cat reports/comparison_*.log
```

Verás tabla como:

Comparativa: Vulnerable vs seguro			
Métrica	Vulnerable	Seguro	Mejora
Tamaño estimado	950 MB	150 MB	-85%
CVEs totales	16	0	-100%
CRITICAL	3	0	
HIGH	8	0	

Usuario	root 	nodejs 	
npm install/ci	install	ci	
Healthcheck	NO	Sí	

Verificación: ¿Los números del reporte coinciden con Docker?

- Vulnerable: 950 MB 
- Seguro: 150 MB 

Paso 2: Analizar npm audit REAL

cat reports/npm_audit_*.log

Verás vulnerabilidades reales:

8 vulnerabilidades encontradas:

HIGH - express 4.17.1

└ Descripción: Muestra información sensible en error responses

HIGH - lodash 4.17.15

└ Descripción: Command Injection en lodash.template()

MEDIUM - qs

└ Descripción: Prototype Pollution en qs.parse()

Paso 3: Entender origen de las 16 vulnerabilidades

Los 16 CVEs viene de:

- npm packages: 8 (express, lodash, qs, body-parser, etc.)

- Node.js 12 (EOL): 4 CVEs conocidas del runtime
- Debian 11.8 (viejo): 4 CVEs del sistema operativo base

¿Por qué con Alpine desaparece todo?

- Alpine 3.20: Sin CVEs conocidas
- Node.js 20-LTS: Sin CVEs conocidas
- npm audit muestra 0 vulnerabilidades en dependencias

Para pensar:

Verifica que entiendes la conexión:

1. ¿Qué viste en Docker?
 - vulnerable: 950 MB, usuario root
 - secure: 150 MB, usuario nodejs
2. ¿Qué dice el reporte?
 - Mismo tamaño, mismo usuario
3. ¿De dónde vienen las vulnerabilidades?
 - Base image (80%), dependencias npm (15%), configuración (5%)
4. ¿Cómo arreglamos todo?
 - Cambiar FROM node:12 → FROM node:20-alpine
 - Cambiar npm install → npm ci
 - Cambiar USER root → USER nodejs